

SeekAndDestroy

An enterprise-grade AI platform that decides where a workload should run across a bank's infrastructure estate — and shows its evidence.

Owner: Praveen Yadav (E1001) Revision: 2 September 2026 Status: In build

The goal. Build a production-grade, industry-best AI system that a tier-1 bank's infrastructure organisation could actually run: it recommends placement and capacity decisions across a live CMDB, justifies every recommendation from real operational history, and is correct by construction rather than by the good behaviour of a language model.

What is being built

An agentic AI platform over an enterprise configuration-management database. An engineer asks, in their own words, where a workload should be hosted — or why a particular cluster was not recommended — and the platform investigates: it reads the estate, scores candidates against capacity, compatibility, cost, resiliency, dependency and risk, retrieves the operational history that bears on the decision, and returns a shortlist with the reasoning attached and a human review step before anything is approved.

It is not a chatbot over documentation, and it is not a dashboard. It is a decision system with an audit trail.

The standard: enterprise grade, production grade

These words are load-bearing. They are defined here so they can be tested rather than asserted.

STANDARD	WHAT IT MEANS CONCRETELY
Enterprise data model	The CMDB is modelled the way ServiceNow models one — a configuration-item table with class inheritance, typed relationships in a relationship table, affected-CI lists on tasks. Any practitioner who administers ServiceNow should recognise the schema on sight.
Real scale	1,200 applications across 256 clusters, 10,000 incidents carrying ~90,000 work notes, 1,000 changes and their outcomes. Enough that ranking, retrieval and forecasting are genuinely exercised rather than demonstrated.
Real models	Real LLM providers throughout. Mock mode exists only as an offline development fallback and is never the operating state, never presented as a working result.
Production posture	Deployed and reachable over TLS; least-privilege database logins with no blanket permissions; authenticated access; audit rows for every consequential action; spend budgets enforced per provider; resumable background jobs that survive restarts.
Measured	Every model call priced in currency and timed as a distribution, budgets enforced in money per provider, evaluation gating the build, and observability switched on in production rather than merely present in the repository.
Verifiable	Every number on screen traceable to a value the system computed, and every qualitative claim traceable to an incident, change or problem record a human can open.

Non-negotiable principles

1. The model explains; Python decides.

No score, cost, utilisation figure or capacity projection is ever produced by a language model. The model reads unstructured text and emits bounded, cited, structured findings; deterministic code maps those findings to numbers. This is enforced at runtime, not by convention — a guard rejects any narration containing a number that did not come from the engine.

Because a recommendation an engineer cannot verify is a recommendation they will not act on, and a plausible wrong number is worse than no number.

2. Evidence over assertion.

Findings carry the identifiers of the records they came from. A low-confidence extraction is treated as no finding rather than as a penalty. Scores are computed on a schedule and persisted with a timestamp, so two runs on the same estate agree.

Reproducibility is what separates a decision system from a generator.

3. Structure is queried; prose is retrieved.

Anything expressible as a filter is answered by SQL and the relationship graph — exactly, and countably. Retrieval exists for what no column can hold: the work notes, resolution notes and root-cause narratives where the nature of a failure actually lives.

Incident nature matters more than incident count, and nature is written in prose, not chosen from a dropdown at closing time.

4. A human approves.

The platform shortlists and argues; it does not place workloads. Every decision records who made it, when, and why — including a decision to reject everything offered.

Accountability for a production placement belongs to a named person.

5. Nothing sensitive leaves the boundary carelessly.

No real employer naming conventions, data-centre names or line-of-business mappings anywhere in the corpus. Credentials live only in ignored environment files. Sending operational prose to a hosted provider is treated as a data-governance decision with a local-model option, not a default.

The platform beneath it

A second goal sits under the first: SeekAndDestroy is also the vehicle for operating and evaluating language models seriously — multi-provider comparison on identical evidence, evaluation harnesses scored on latency, cost and output quality, prompt versioning, per-model telemetry and per-provider spend control. The recommendation product is the workload that makes those requirements real.

LAYER	IMPLEMENTATION
Reasoning	Agentic graph pipeline with explicit nodes for requirement extraction, candidate selection, rule evaluation, scoring, retrieval, explanation and human review
Retrieval	Hybrid — dense vectors alongside sparse BM25, fused by reciprocal rank; chunked so a ticket becomes header, work notes and resolution rather than one averaged vector
Record of truth	SQL Server; ServiceNow-shaped CMDB with typed CI relationships; ITSM history of incidents, changes and problems
Delivery	Python service, .NET gateway, React interface; containerised and deployed to cloud infrastructure behind a tunnel with TLS
Governance	Audit log, decision records, index-run history, spend budgets, least-privilege data access

Observability, cost and evaluation

These are treated as product requirements, not instrumentation added afterwards. The standard is one sentence: **every model call is a measured, priced, reproducible event, and every quality claim is backed by a test that is capable of failing.** A system that cannot say what it costs and cannot prove it has not regressed is a proof of concept regardless of what it does well.

Telemetry — what every model call must emit

DIMENSION	REQUIRED
Identity	Provider, model, model version, graph node, tool, investigation id, acting employee
Volume	Prompt tokens, completion tokens, cached tokens, whether the input was truncated
Cost	Unit price in force at the moment of the call, computed cost, currency — <i>persisted on the row</i> , never re-derived later from a price that may since have changed
Latency	Queue wait, provider time and total, as histograms — p50, p95, p99 per model, not averages and not counters
Outcome	Success or error class, retry count, fallback from and to, cache hit or miss

Full-fidelity retention for the operational window, aggregated beyond it. Every metric has a named owner, a threshold and an alert; no dashboard panel exists without a corresponding alert rule, because a chart nobody is paged on is decoration.

Cost control — denominated in money, not in calls

- A price table per provider and model, with effective dates, so historical cost stays correct after a vendor changes pricing.
- Budgets expressed in currency and enforced **per provider** as well as per workload, so one model cannot consume another's allowance.
- Daily and monthly ceilings with a soft warning before a hard stop, and the denial recorded as telemetry rather than only as an exception.
- Cost per investigation attributable to the investigation, and month-end spend projected from actuals.

Evaluation — every case must be capable of failing

- **Deterministic graders gate the pipeline.** Number fidelity, entity fidelity and completeness are arithmetic, not opinion, and a failure blocks the result rather than annotating it.
- **An LLM judge is used only where determinism cannot see** — tone, coherence, whether the explanation answers the question asked. It has no numeric authority and no dimension in which to express one.
- **A golden set broad enough to be evidence**, covering every investigation type, including adversarial cases, negative cases where the correct answer is a refusal, and cases whose correct answer changes when the estate changes.
- **Retrieval measured, not asserted** — recall@k, MRR and NDCG, with dense, sparse and hybrid reported separately so a hybrid gain is attributable. Cases are selected so that a wrong retriever scores worse; a case that cannot fail is a case that measures nothing.

- **Multi-model comparison as a first-class capability** — identical evidence across N providers, scored on output quality, latency and cost, produced by one command as one report. This is the difference between a platform that operates models and an application that calls one.
- **Regression gating.** Evaluation runs in continuous integration and a drop beyond threshold fails the build.
- **Drift detection.** A fixed set re-run on a schedule against unchanged data, alerting on movement — because the provider changes underneath us without notice.

How we will know it is done

- An infrastructure engineer who works in ServiceNow daily reads the schema and recognises it, without explanation. – *recognition, not tolerance*
- Every recommendation on screen can be traced to the rule that eliminated a candidate and the records that justified the winner. – *zero unexplained rankings*
- Retrieval quality is measured, not claimed: recall@k, MRR and NDCG on a golden set whose cases can actually fail. – *published numbers*
- Resiliency reflects topology, not node count — an application on four VMs sharing two physical hosts is reported as two-way redundant. – *graph-derived*
- The system answers "what breaks if this host dies" by traversing real dependencies. – *blast radius with depth*
- It runs unattended: indexing resumes after a restart, budgets hold, and no run silently half-completes. – *survives a reboot*
- Every approval carries a name, a timestamp and a reason. – *complete audit trail*
- The platform can state what any investigation cost, in currency, broken down by model. – *cost is a stored fact*
- Latency is answerable per model at p95, not as an average and not by hand from timestamps. – *histograms, not counters*
- Spend ceilings are enforced in money and per provider, with the denial recorded. – *no shared allowance*
- The same evidence can be run across every configured model and compared on quality, latency and cost in one report. – *one command*
- Evaluation runs in CI and a regression fails the build rather than being noticed later. – *gated, not observed*
- Observability is switched on in production, with an alert behind every panel. – *running, not merely built*

Explicit non-goals

- **Not a demo.** Nothing is built to look good in a five-minute walkthrough and fall over on the sixth question.
- **Not a mock.** Simulated providers, synthetic scores and placeholder narration are development scaffolding, never deliverables.
- **Not an autonomous actor.** The platform does not provision, migrate or decommission anything. It recommends, and a person decides.
- **Not a general assistant.** Questions outside this estate are refused clearly, with examples of what can be asked instead.